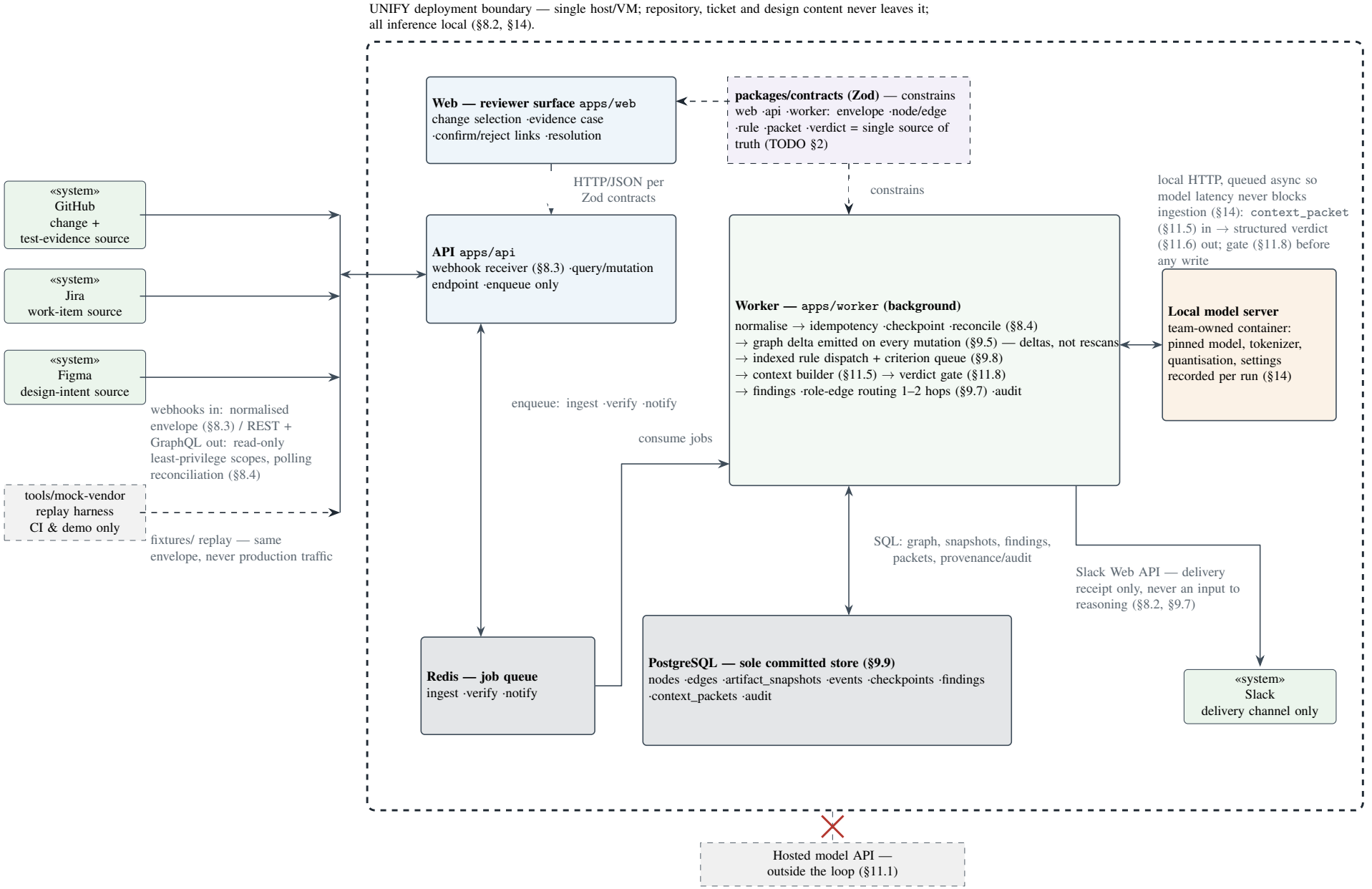

UNIFY deployment boundary — single host/VM; repository, ticket and design content never leaves it; all inference local (§8.2, §14).
«system» GitHub change + test-evidence source
«system» Jira work-item source
«system» Figma design-intent source
tools/mock-vendor replay harness CI & demo only
webhooks in: normalised envelope (§8.3) / REST + GraphQL out: read-only least-privilege scopes, polling reconciliation (§8.4)
fixtures/ replay — same envelope, never production traffic
Web — reviewer surface apps/web
change selection ·evidence case ·confirm/reject links ·resolution
packages/contracts (Zod) — constrains web ·api ·worker: envelope ·node/edge ·rule ·packet ·verdict = single source of truth (TODO §2)
HTTP/JSON per Zod contracts
constrains
API apps/api
webhook receiver (§8.3) ·query/mutation endpoint ·enqueue only
Worker — apps/worker (background)
normalise → idempotency ·checkpoint ·reconcile (§8.4)
→ graph delta emitted on every mutation (§9.5) — deltas, not rescans
→ indexed rule dispatch + criterion queue (§9.8)
→ context builder (§11.5) → verdict gate (§11.8)
→ findings ·role-edge routing 1–2 hops (§9.7) ·audit
local HTTP, queued async so model latency never blocks ingestion (§14): context_packet (§11.5) in → structured verdict (§11.6) out; gate (§11.8) before any write
Local model server
team-owned container: pinned model, tokenizer, quantisation, settings recorded per run (§14)
enqueue: ingest ·verify ·notify
consume jobs
SQL: graph, snapshots, findings, packets, provenance/audit
Slack Web API — delivery receipt only, never an input to reasoning (§8.2, §9.7)
Redis — job queue
ingest ·verify ·notify
PostgreSQL — sole committed store (§9.9)
nodes ·edges ·artifact_snapshots ·events ·checkpoints ·findings ·context_packets ·audit
«system» Slack delivery channel only
Hosted model API — outside the loop (§11.1)